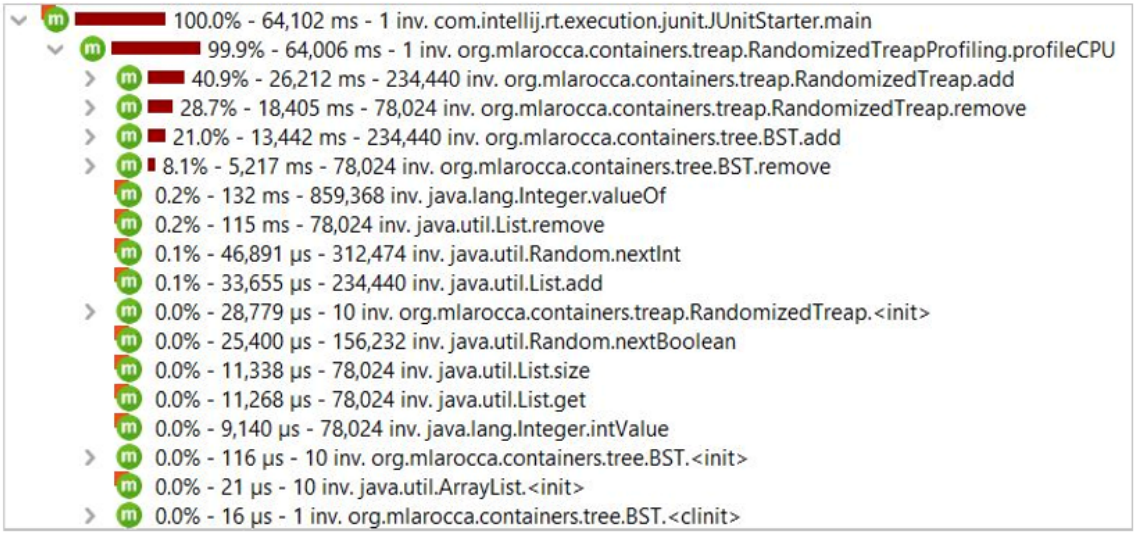


Figure 3.20 Profiling CPU usage when inserting/removing random unbounded integers in a BST and randomized treap.

From this first test, it seems that we pay a high price, in the general case, for the overhead due to the greater complexity of randomized treaps. This result is somehow expected, because when the height of the trees is approximately the same, the code for treaps is substantially more complex than BST's.

Should we just throw away our `RandomizedTreap` class? Well, not so fast. Let's see, for instance, what happens when profile the second test case introduced in section 3.5.2, the one where we still add random integers to the containers, but limited to the range [0, 1000].

In the previous section, we saw that, in this case, BST's height grows linearly, while for randomized treaps we still have a logarithmic growth.